

INSTITUTO FEDERAL DE
EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
PARAÍBA
Campus João Pessoa

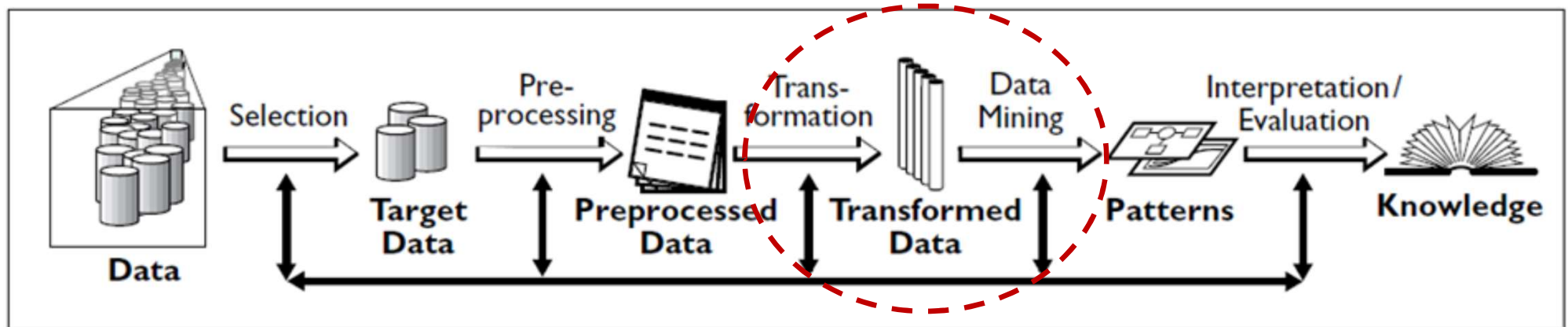
Atividades de pré-processamento

Transformação de Dados

Prof. Alex Cunha
alex@ifpb.edu.br

Atualizada em 30/03/2026

Visão geral do Processo de KDD



Objetivo:

Etapa do pré-processamento em que as **variáveis originais** são modificadas, reescaladas ou convertidas para uma nova representação, mais adequada à **análise** ou ao **modelo de aprendizado de máquina**

Recordando

- ▶ Variáveis numéricas

- ▶ **Variável Contínua:** variáveis numéricas que podem assumir infinitas possibilidades dentro de um **intervalo real**

- ▶ Exemplo: altura (1.65m), peso (65.2 kg), renda (R\$ 8.300,00)

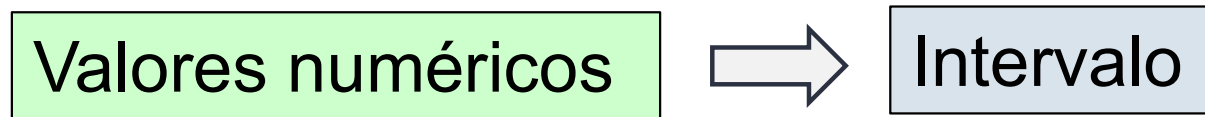
- ▶ **Variável Discreta:** variáveis numéricas que assumem **valores contáveis**, normalmente valores inteiros finitos (sem valores intermediários)

- ▶ Exemplo: idade (52 anos), num_cliques (198), numero_de_quartos (3 quartos)



Discretização

- Processo de transformação de valores de **variáveis contínuas (numéricas)** em valores discretos, dividindo-as em intervalos ou “grupos” **grupos ou faixas discretas (bins ou caixas)**



Idade

15	23	35	44	36	65	51
----	----	----	----	----	----	----



jovem	0-17
Adulto	18-59
Idoso	60+

Discretização

► Outros exemplos

Dados de dependentes de funcionários (intervalo de valores de 0 a 15)

NumDep	0 a 2	3 a 4	5 a 15
categórico	padrão	aumentado	elevado
numérico	0	1	2

Dados de desempenho de estudantes (intervalo de valores das notas de 0 a 10)

Resultado:

Não-Aprovado se valor < 5

Aprovado Regular se $5 \leq \text{valor} < 7$

Aprovado Bom se $7 \leq \text{valor} \leq 9$

Aprovado Excelente se valor > 9

Discretização

- ▶ **Binning:** técnica de discretização que consiste em dividir uma variável numérica contínua (ou discreta) em um conjunto de **faixas discretas**, denominadas de “**bins**” (caixas)
 - ▶ Intervalos fixos (**equal-width binning**)
 - ▶ Intervalos baseado na frequência (**equal-Frequency binning**)
 - ▶ Baseado em regras do negócio: mais controlada



Discretização - Métodos

- ▶ **Binning de Largura igual (automática)**
 - ▶ Divide o intervalo de valores em faixas do mesmo tamanho
 - ▶ **E.g. :** Notas de 0 a 100 : 3 bins
 - ▶ **Cada bin terá um intervalo de 33,33**

bin 1
[0,33.33[

bin 2
[33.33, 66.67[

bin 3
[66.67, 100]



Discretização - Métodos

► Binning de Largura igual (automática)

```
# Dados contínuos de exemplo
```

```
valores = np.array([5, 6, 5, 7, 12, 17, 18, 24, 33, 45, 59, 63, 70])  
df = pd.DataFrame({'idade': valores})
```

```
► # 1. Equal-width binning (3 faixas de mesma largura)  
df['idade_bin_width'] = pd.cut(df['idade'], bins=3,  
                               labels=["Jovem",  
                                       "Meia-idade",  
                                       "Idoso"],)  
  
print("\n=== Equal-width Binning ===")  
print(df[['idade', 'idade_bin_width']])  
  
print(f'\nFaixas de valores dos bins:')  
print(df['idade_bin_width'].value_counts())
```

=== Equal-width Binning ===

	idade	idade_bin_width
0	5	Jovem
1	6	Jovem
2	5	Jovem
3	7	Jovem
4	12	Jovem
5	17	Jovem
6	18	Jovem
7	24	Jovem
8	33	Meia-idade
9	45	Meia-idade
10	59	Idoso
11	63	Idoso
12	70	Idoso

Mínimo : 5

Máximo: 70

Range: 70 – 5 = 65

Largura do bin: $65 / 3 = 21.7$

Bin 1: [5, 26.67[

Bin 2: [26.67, 48.33[

Bin 3: [58.33, 70]



Mas pode quebrar faixas lógicas

Discretização - Métodos

► Binning de Largura igual (automática)

Quando aplicar?

- Quando o atributo numérico possui **distribuição aproximadamente uniforme** e **não existem pontos de corte no domínio do atributo** que tenham um significado intrínseco

Dia	Temperatura
1	24 (B1)
2	27 (B1)
3	28 (B1)
4	28 (B1)
5	30 (B2)
6	31 (B2)

Dia	Temperatura
7	31 (B2)
8	32 (B2)
9	33 (B3)
10	34 (B3)
11	35 (B3)
12	36 (B3)

Mínimo : 24

Máximo: 36

Range: $36 - 24 = 12$

nBins = $12/3 = 4$

Bin 1: (23.9, 28.0]

Bin 2: (28.0, 32.0]

Bin 3: (32.0, 36.0]

- A temperatura **tem dados distribuídos**, é **contínua** e **sem cortes naturais rígidos**

Discretização - Métodos

- ▶ **Binning de Frequência igual (automática)**
 - ▶ Divide os dados de maneira que cada faixa tenha aproximadamente **o mesmo número de observações**
 - ▶ **E.g.:** Quartis (Q1, Q2, Q3), Decis, percentis

```
# Dados contínuos de exemplo
valores = np.array([5, 6, 5, 7, 12, 17, 18, 24, 33, 45, 59, 63, 70])
df = pd.DataFrame({'idade': valores})
```

```
# 2. Equal-frequency binning (quantis)
df['idade_bin_quantis'] = pd.qcut(df['idade'], q=3, labels=["Baixa",
                                                         "Média",
                                                         "Alta"])

print("\n=== Equal-frequency Binning (Quantis) ===")
print(df[['idade', 'idade_bin_quantis']])
```

Bin 1: 4 ou 5 elementos

Bin 2: 4 ou 5 elementos

Bin 3: 4 ou 5 elementos



Bins equilibrados



Mas pode quebrar faixas lógicas

```
=== Equal-frequency Binning ===
              (Quantis)
idade idade_bin_quantis
0         5             Baixa
1         6             Baixa
2         5             Baixa
3         7             Baixa
4        12             Média
5        17             Média
6        18             Média
7        24             Média
8        33             Alta
9        45             Alta
10       59             Alta
11       63             Alta
12       70             Alta
```

Discretização - Métodos

► Binning de Frequência igual (automática)

Quando aplicar?

- Quando o atributo numérico apresenta **distribuição assimétrica (skewed)** ou contém **outliers**, e que **não existem pontos de corte no domínio do atributo**

ID	Renda (R\$)
1	1200 (B1)
2	1500 (B1)
3	1800 (B1)
4	2000 (B1)
5	2200 (B2)
6	2500 (B2)

ID	Renda (R\$)
7	3000 (B2)
8	3500 (B2)
9	4000 (B3)
10	4300 (B3)
11	4500 (B3)
12	10000 (B3)

Outlier = 10.000

Bin 1: 1200-2000

Bin 2: 2200-3500

Bin 3: 4000-10000

Ganha em estatística,
mas perde em
semântica

- Evita **bins** com poucos exemplos. **Renda** é um atributo normalmente **assimétrico**

Discretização - Métodos

► Binning Manual

Baseado em regras do negócio/domínio
(supervisionada por domínio)

```
# Dados contínuos de exemplo
valores = np.array([5, 6, 5, 7, 12, 17, 18, 24, 33, 45, 59, 63, 70])
df = pd.DataFrame({'idade': valores})
```

```
def faixa_idade(idade):
    if idade <= 12:
        return 'Criança'
    elif idade <= 17:
        return 'Adolescente'
    elif idade <= 59:
        return 'Adulto'
    else:
        return 'Idoso'

df['idade_manual'] = df['idade'].apply(faixa_idade)
print("\n=== Discretização Manual ===")
print(df[['idade', 'idade_manual']])
```

```
=== Discretização Manual ===
idade idade_manual
0      5      Criança
1      6      Criança
2      5      Criança
3      7      Criança
4     12      Criança
5     17  Adolescente
6     18      Adulto
7     24      Adulto
8     33      Adulto
9     45      Adulto
10    59      Adulto
11    63      Idoso
12    70      Idoso
```

⚠ Intervalos são criados por um especialista de domínio

Discretização

► Vantagens

- **Redução de ruído:** pequenas variações nos dados contínuos são absorvidos pelos intervalos
- **Tratamento de outliers:** ajuda a atenuar o efeito de *outliers*, colocando-os em intervalos extremos
- **Interpretabilidade dos dados:** melhora a compreensão por humanos (e.g. “faixa etária” é mais interpretável do que a idade exata)
- **Numérico/Categórico:** Permite que valores numéricos possam ser mapeados para atributos nominais (categóricos)



Discretização

► Desvantagens

- **Perda de informação e precisão:** perde-se a granularidade e variabilidade dos dados
 - *Agrupamento de valores distintos em **bins** faz com que se perca a variação detalhada dentro de cada grupo*
- **Limites mal definidos:** escolher poucos intervalos pode simplificar demais e esconder padrões. Muitos intervalos podem trazer maior complexidade
- **Descontinuidade:** a discretização transforma relações contínuas em “saltos” (jumps) entre intervalos, o que pode representar erroneamente um fenômeno que muda de forma contínua



Discretização

► O problema da descontinuidade



Uma mudança contínua e pequena na entrada (idade) **gera uma mudança discreta e drástica na saída** (classificação de risco), quebrando a relação linear entre os dados

Outliers

Definição depende do problema

- ▶ “São valores que se distanciam significativamente da maioria dos dados”
- ▶ Entender outliers é fundamental em uma análise de dados: *distorcem análises e podem comprometer um modelo*
 - ▶ Por quê?
 - I. Os outliers podem **enviesar** negativamente todo o resultado de uma análise
 - Diversidade, justiça de dados
 - II. O comportamento dos outliers pode ser justamente **o que se está procurando**.
 - **Qual despesa está fora do padrão?**



Outliers

Definição depende do problema

► Considerações:

- **Impactam a média:** podem inflar ou reduzir significativamente a média (perde representatividade)
- **Nem sempre são erros:** podem ser dados reais de alto valor que exigem investigação
- Podem ser detectados por meio de gráficos (boxplots e histogramas) ou método IQR



Outliers ou anomalias

Em nível de **coluna/atributo** - **Valores extremos**

Funcionário	Função	Salário
Joana Braga	Analista de TI	R\$ 5.600,00
Valter Lopes	Analista de TI	R\$ 10.600,00
Ana Moreira	Analista de TI	R\$ 105.800,00

Média salarial: R\$ 40.666,7



O que fazer?

1. **Remover** o valor extremo?
2. **Tratar** os valores? E.g., via normalização



Outliers ou anomalias

Em nível de **instância** – saem completamente do padrão da amostra

ENEM

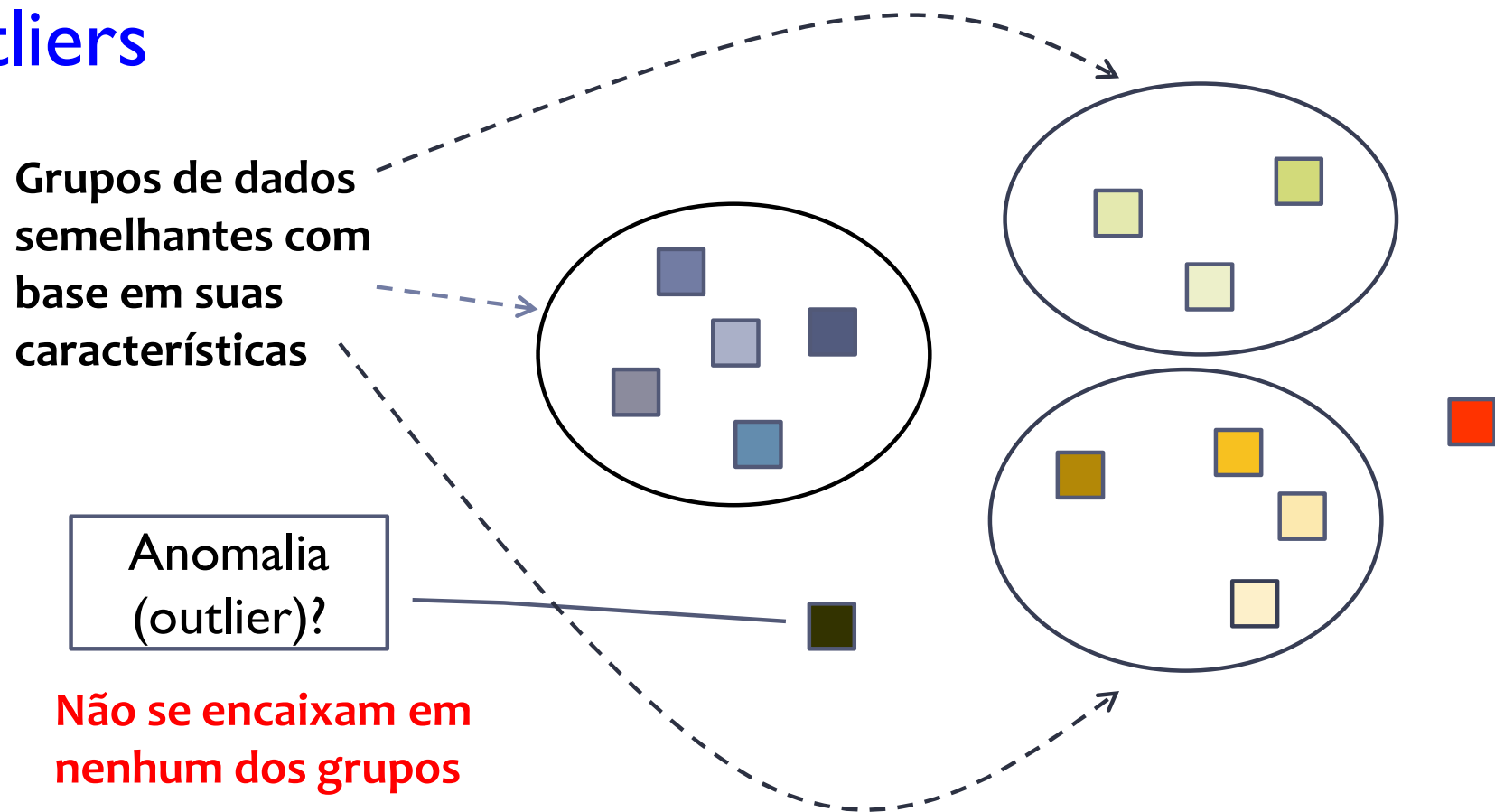
Candidato	Renda familiar	Escolaridade da mãe	Gestão do tempo	Nota no ENEM
1	Alta	Graduação	Muita	750
2	Baixa	Graduação	Nenhuma	460
3	Baixa	Fundamental	Nenhuma	800
4	Média	Médio	Pouca	600

Quem é o outlier?



Outliers em nível de instância

► Clusterização/agrupamento para identificar outliers



** Aprendizado de máquina não supervisionado

Outliers em colunas

▶ Como são introduzidos?

- ▶ Erros de medição ou de entrada
- ▶ Variações naturais do processo em que os dados foram coletados
- ▶ Eventos raros ou anomalias

▶ Exemplos

- ▶ Em um conjunto de idades com alturas de adultos (média = 1,75), uma altura de **2,20** m pode ser um outlier.
- ▶ Em um conjunto de dados com lançamentos de vendas mensais (e.g., média de **30.000** por mês), um mês com faturamento de **300.000** pode ser um outlier decorrente de erro ou sazonalidade extrema



Outliers

- ▶ Outliers nem sempre são “erros”; às vezes **representam diversidade**
- ▶ Em um contexto, pode representar um **grupo minoritário** ou **uma exceção legítima** que merece ser compreendida, e não descartada

Cenário do Problema:

“Na análise salarial de um dataset contendo dados de empregados, observa-se que há **poucas mulheres** ou **pessoas negras** ocupando cargos executivos com altos salários”. Os salários podem parecer outliers, mas são **casos importantes de diversidade**.



Remover os outliers pode afetar a representatividade desses grupos



Outliers

- ▶ Decisões éticas na **remoção de outliers**
 - ▶ Esse valor é um **erro** ou uma **exceção significativa**?
 - ▶ Ao remover a informação, estarei apagando a presença de algum grupo social (diversidade)?

Grupos minoritários podem não ter representação

- ▶ Estarei reforçando desigualdades ao filtrar dados fora do padrão? (justiça)

Em um problema de crédito, remover todos os outliers de **renda ou escolaridade**, pode ignorar casos reais de ascensão social



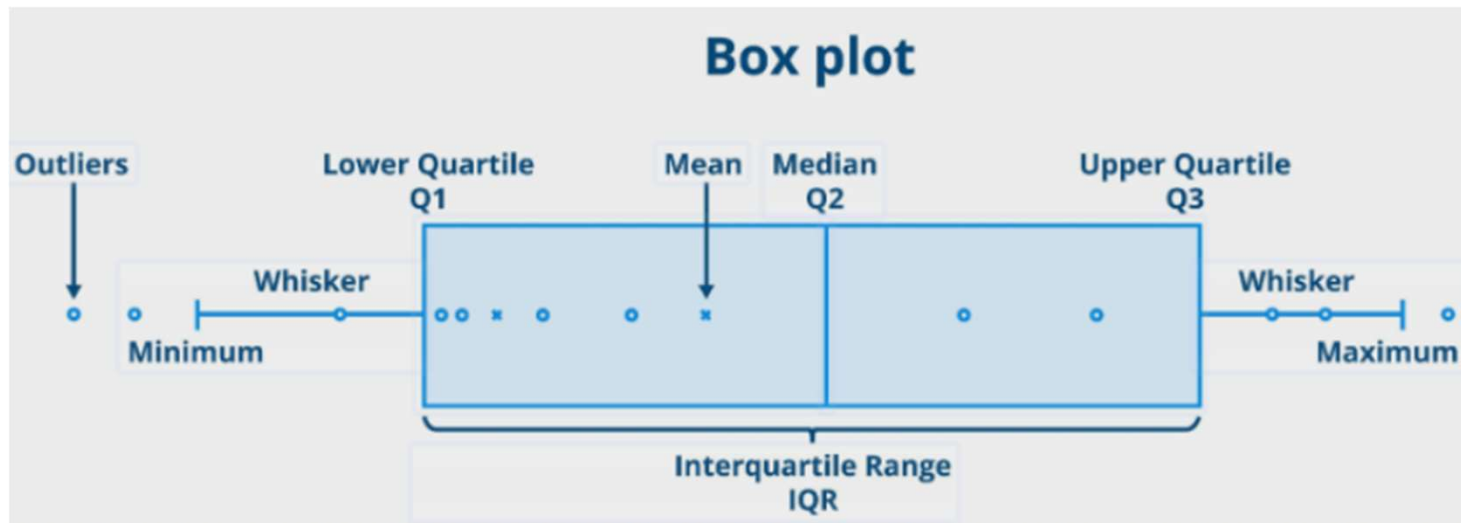
Outliers

- ▶ Método IQR (Intervalo Interquartil)
 - ▶ Um valor é considerado **outlier** se estiver **abaixo do limite inferior** ou **acima do limite superior**

$IQR = Q3 - Q1$ (diferença entre o terceiro e o primeiro quartil)

$LI = Q1 - 1.5 \times IQR$

$LS = Q3 + 1.5 \times IQR$



Outliers

► Método IQR (Intervalo Interquartil)

```
data = [10, 12, 15, 18, 20, 22, 25, 28, 30, 90, 150]
```

Média: 38.18

Mediana: 22.00

Desvio Padrão: 43.07

Q1: 16.5,

Q3: 29.0

IQR: 12.5

Limite Inferior: -2.25

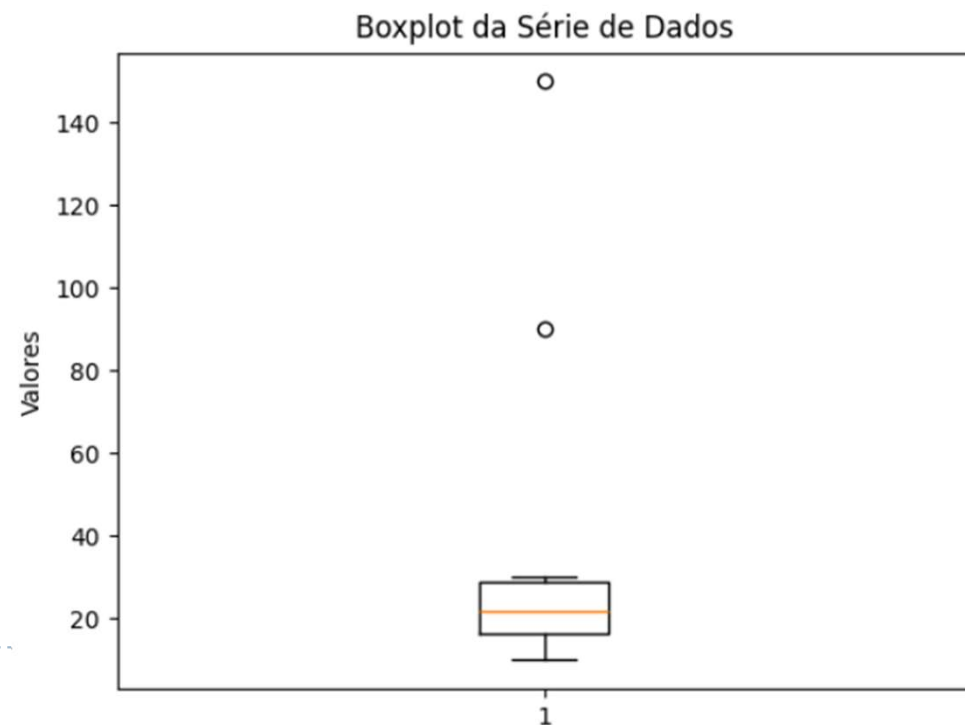
Limite Superior: 47.75

```
Outliers identificados:
```

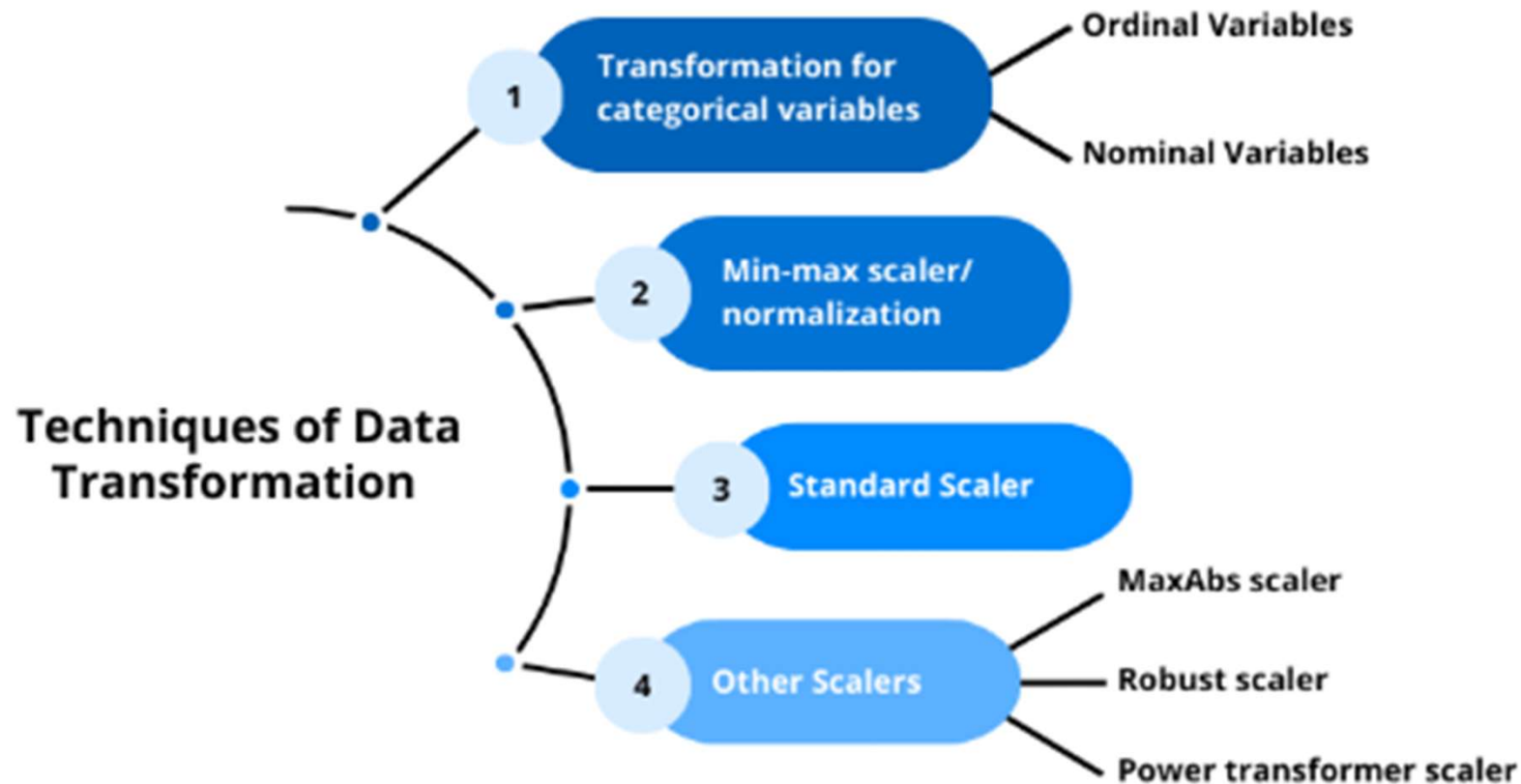
```
9      90
```

```
10     150
```

```
dtype: int64
```



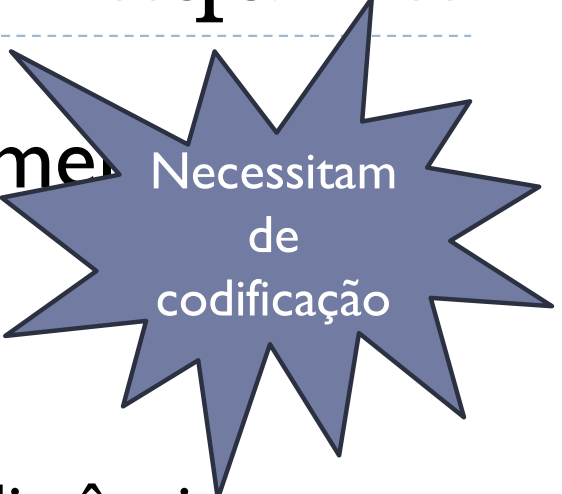
Transformação de Dados



Alteram o formato dos dados para que possam ser utilizados por algoritmos de aprendizado de máquina

Dados para Aprendizado de Máquina

- ▶ Muitos algoritmos de AM funcionam melhor com **dados numéricos**



Necessitam
de
codificação

- ▶ KNN – calcula a distância entre amostras
- ▶ K-means – agrupa amostras com base na distância entre elas
- ▶ Support Vector Machines (SVM) - utiliza vetores de suporte e funções do kernel
- ▶ Regressão linear – relação entre variáveis (Dep/Indep)
- ▶ Menos sensível à codificação
 - ▶ Árvore de decisão (numéricos ou categóricos)



Dados para Aprendizado de Máquina

- ▶ Redes Neurais, SVM, regressão – precisam de **entrada numérica padronizada/normalizada**
 - ▶ Não trabalham diretamente com texto, imagem, categoria
 - ▶ Os valores numéricos precisam ser adaptados para **uma escala apropriada**
- ▶ Técnicas: **Normalização, Padronização e Escalador Robusto**



Normalização

- ▶ Minimiza problemas oriundos do uso de unidades e dispersões distintas entre as variáveis
- ▶ Redimensiona os **valores dos dados em um intervalo especificado**, como, por exemplo: -1,0 a 1,0 ou **0,0 a 1,0** (mais comum)
- ▶ E.g., **Min-Max no intervalo [0,1]**

$$x_{\text{norm}} = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

Sensibilidade a outliers:  Muito alta

Evita que variáveis com **valores elevados** dominem o treinamento



Dados para Aprendizado de Máquina

► Normalização

Antes

potencia_cv	idade	preco
100	1	35000
150	3	95000
200	5	120000
250	7	180000
300	9	250000

Depois

potencia_norm	idade_norm	preco_norm
0.00	0.00	0.000000
0.25	0.25	0.279070
0.50	0.50	0.395349
0.75	0.75	0.674419
1.00	1.00	1.000000

Compare o valor da potência, da idade e do preço. Estão mais próximos?

MinMaxScaler()

```
import numpy as np
import pandas as pd
from sklearn.preprocessing import MinMaxScaler

dados = {
    'potencia_cv': [100, 150, 200, 250, 300],
    'idade': [1, 3, 5, 7, 9],
    'preco': [35000, 95000, 120000, 180000, 250000]
}
df = pd.DataFrame(dados)

scaler = MinMaxScaler()
X = df[['potencia_cv', 'idade', 'preco']]
X_normalizado = scaler.fit_transform(X)

df_normalizado = pd.DataFrame(X_normalizado,
                               columns=['potencia_norm',
                                         'idade_norm',
                                         'preco_norm'])

df_final = pd.concat([df, df_normalizado], axis=1)

# Exibindo a tabela final
print(df_final)
```

Normalização

▶ Vantagens

- ▶ Garante que **todos os dados** estejam na **mesma escala**, (facilita a convergência em algoritmos de otimização)

▶ Desvantagens

- ▶ **Extremamente sensível a outliers**, pois valores muito altos ou baixos distorcem a escala

Sensibilidade a outliers:  Muito alta

Ignora a forma de distribuição dos dados, pois apenas **reescala** os dados para o intervalo definido



Padronização

- ▶ Técnica que transforma os dados para que tenham **média 0 e desvio padrão igual a 1**
 - ▶ **Média = 0**: centraliza os dados em torno da origem do eixo (0)
 - ▶ **DevPad = 1**: uniformiza a dispersão das variáveis
- ▶ E.g., **Fórmula Z-Score**

$$x_{\text{padronizado}} = \frac{x - \mu}{\sigma}$$

média (μ)
desvpad (σ)

Facilita a visualização (**gráficos**) e cálculos vetoriais e evita que uma variável com **maior variância** domine outras com variâncias menores

Dados para Aprendizado de Máquina

► Padronização

Antes

	potencia_cv	idade	preco
0	100	1	35000
1	150	3	95000
2	200	5	120000
3	250	7	180000
4	300	9	250000

Depois

potencia_norm	idade_norm	preco_norm
-1.414214	-1.414214	-1.372658
-0.707107	-0.707107	-0.557217
0.000000	0.000000	-0.217451
0.707107	0.707107	0.597989
1.414214	1.414214	1.549336

Fórmula Z-score de padronização

$$x_{\text{padronizado}} = \frac{x - \mu}{\sigma}$$

média (μ) = 200

desvpad (σ) = 70.71

```
scaler = StandardScaler()
```

A variável agora está com média ≈ 0 e desvio padrão ≈ 1

$$X_{100} = (100 - 200)/70.71 = -1.41$$

Dados para Aprendizado de Máquina

► Padronização

Antes

Altura (cm)	Peso (kg)	Idade
180	80	25
160	65	45
170	70	35

Depois

Altura (cm)	Peso (kg)	Idade
-1.22	1,34	1.22
0	-1,07	-1.22
1.22	-0,27	0

Padronização Z-score

```
from sklearn.preprocessing import StandardScaler
dados = {
    'potencia_cv': [100, 150, 200, 250, 300],
    'idade': [1, 3, 5, 7, 9],
    'preco': [35000, 95000, 120000, 180000, 250000]
}
df = pd.DataFrame(dados)

scaler = StandardScaler()
X = df[['potencia_cv', 'idade', 'preco']]
X_normalizado = scaler.fit_transform(X)

df_normalizado = pd.DataFrame(X_normalizado,
                               columns=['potencia_norm', 'idade_norm',
                                         'preco_norm'])

# Juntando original + normalizado para comparar
df_final = pd.concat([df, df_normalizado], axis=1)

# Exibindo a tabela final
print(df_final)
```

Padronização

▶ Vantagens

- ▶ Mantém **valores negativos**, o que pode ser útil para algoritmos que interpretam o sinal dos dados

▶ Desvantagens

- ▶ **Pode** ser **sensível a outliers**, pois a **média** e o **desvio padrão** são influenciados por valores extremos.

Sensibilidade a outliers:  Moderada/alta

Não exige que os dados sejam **normalmente distribuídos**, mas **proporciona melhor representação** quando são **aproximadamente normais**



Escalonador

► Robust Scaler

- Método de escalonamento de dados eficaz quando o conjunto de dados contém **outliers discrepantes**.

- **Estatística:**

$$x_{scaled} = (x - \text{mediana}) / (Q3 - Q1)$$

- Na prática, o outlier ficará **isolado mais longe**, sem “achatar” os demais

“**Robusta**”: como utiliza a **mediana** e o **IQR**, os outliers não influenciam as estatísticas de escalonamento tanto quanto fariam a média e o desvio padrão

Escalonador

► Robust Scaler x Padronização x Min-Max

Data = [10.0, 12.0, 11.0, **100.00**, 9.0]

Média: 28.4
Mediana: 11.0
Desvio Padrão: 35.81

```
MinMax Scaler  
[[0.01098901]  
 [0.03296703]  
 [0.02197802]  
 [1.         ]  
 [0.         ]]
```

O que mais sofre. o outlier **100** se torna **1.0**, e o 9 se torna **0.0**. A distância do 12 para o 100 é grande, de maneira que os demais valores fiquem muito próximo do zero (e.g. 0.01, 0.02)

```
Standard Scaler  
[[-0.51376609]  
 [-0.45792195]  
 [-0.48584402]  
 [ 1.99922021]  
 [-0.54168816]]
```

O outlier **100** puxa a **média para cima e aumenta drasticamente o desvio padrão**. Os valores 9, 10, 11 e 12 ficarão “**espremidos**” em valores muito próximos, o que vai dificultar o algoritmo distingui-los

```
Robust Scaler  
[[-0.5]  
 [ 0.5]  
 [ 0. ]  
 [44.5]  
 [-1. ]]
```

A mediana faz com que a maioria dos dados continue espalhada de forma útil, e **o outlier ficará isolado com maior distância**, sem achatar demais os outros valores

Escalonador

▶ Conclusão Analítica

- ▶ **MinMax** → adequado apenas quando não há outliers relevantes
- ▶ **Z-score** → útil quando dados são aproximadamente normais
- ▶ **Robust Scaler** → melhor escolha em cenários com:
 - ▶ Ruído
 - ▶ Caudas pesadas
 - ▶ Outliers extremos



Escalonamento não remove outliers! Ele apenas muda sua influência relativa.



Conversão de atributos categóricos

► Label Encoding

- Atribui um número único a cada categoria
- Simples, eficiente, mas pode **induzir relações ordinais inexistentes**

```
from sklearn.preprocessing import LabelEncoder  
  
sentimento = ['feliz', 'triste', 'feliz', 'triste', 'triste', 'normal', 'ansioso', 'amoroso']  
encoder = LabelEncoder()  
encoded_data = encoder.fit_transform(sentimento)
```

```
print(encoded_data) # Exemplo: [2 4 2 4 4 3 1 0]
```

```
[2 4 2 4 4 3 1 0]
```

Um modelo pode interpretar que **triste** é 2 x maior do que **feliz**, o que não faz sentido

- Modelos de árvore lidam bem com essa abordagem

Conversão de atributos categóricos

► Label Encoding

► Quando não usar **label encoding**?

Quando se tem variáveis nominais, não utilizar **label encoding** especialmente se estiver usando os algoritmos **SVM, KNN, Redes Neurais, Regressão Linear e Regressão Logística**

ALTERNATIVA: One-hot encoding



Conversão de atributos categóricos

▶ One-Hot Encoding

- ▶ “Técnica que transforma uma variável categórica de **n** valores em **n** colunas binárias (0 ou 1), com uma só coluna assumindo o valor **1**”
- ▶ Evita que um modelo interprete ordem/hierarquia entre categorias (quando não existe)
- ▶ Desvantagens
 - ▶ **Pode gera alta dimensionalidade** quando se tem datasets e colunas com muitas categorias



Modelos de árvore **também** lidam bem com essa abordagem



Conversão de atributos categóricos

► One-Hot Encoding

```
import pandas as pd

# Dados de entrada com uma coluna categórica
df = pd.DataFrame({
    'Tipo_Veiculo': ['Sedan', 'SUV', 'Hatch', 'SUV', 'Sedan'],
    'Idade': [45, 34, 23, 40, 29]
})

# Aplicando o One-Hot Encoding
df_encoded = pd.get_dummies(df, columns=['Tipo_Veiculo'], dtype=int)

# Exibindo o resultado
print(df_encoded)
```

	Idade	Tipo_Veiculo_Hatch	Tipo_Veiculo_SUV	Tipo_Veiculo_Sedan
0	45	0	0	1
1	34	0	1	0
2	23	1	0	0
3	40	0	1	0
4	29	0	0	1



Conversão de atributos categóricos

▶ **Ordinal Encoding**

- ▶ Técnica que transforma variáveis categóricas em uma sequência de valores numéricos inteiros **preservando o relacionamento ordinal**
 - ▶ **Não deve ser utilizado para atributos nominais**
- ▶ A ordem deve ser garantida estabelecendo a sequência desejada no código
- ▶ **Desvantagem**
 - ▶ Pode distorcer o significado **quando não houver ordem real**
- ▶ **Quando usar?**
 - ▶ Variável categórica com ordem lógica (menor < meio < maior)



Conversão de atributos categóricos

► Ordinal Encoding

```
from sklearn.preprocessing import OrdinalEncoder
import pandas as pd

df = pd.DataFrame({'escolaridade': ['fundamental', 'médio', 'superior', 'médio',\
                                   'superior', 'superior', 'fundamental']})

# garanta a ordem estabelecendo a sequência
encoder = OrdinalEncoder(categories=[['fundamental', 'médio', 'superior']])
df['escolaridade_encoded'] = encoder.fit_transform(df[['escolaridade']])

print(df)
```

	escolaridade	escolaridade_encoded
0	fundamental	0.0
1	médio	1.0
2	superior	2.0
3	médio	1.0
4	superior	2.0
5	superior	2.0
6	fundamental	0.0